

CERA Medical Product Requirements Document

Full platform requirements delivery plan and engineering workflow

Prepared for CERA Medical and the three-developer delivery team

Document	Value
Owner	Stack & Scale for CERA Medical
Version	1.0
Date	17 September 2026
Status	Approved implementation baseline subject to CERA content and release approval

This document defines the complete first production release of the CERA Medical platform. It fixes the product boundary, user journeys, feature requirements, architecture, data contracts, engineering controls, ten-day delivery sequence, launch criteria, and operating responsibilities. A feature is complete only when its acceptance evidence passes in staging and the required CERA and technical approvals are recorded.

Document map

Part	Purpose
Product definition	Goals, users, scope, decisions, journeys, and success measures
Requirements	Feature catalogue, acceptance criteria, data model, and nonfunctional requirements
Technical delivery	Architecture, repository structure, environments, GitHub controls, CI/CD, testing, and secrets
Execution	Phases, ten-day integrated plan, developer ownership, release, rollback, handover, and risks
References	Official sources reviewed for the September 2026 baseline

1 Product definition

1.1 Product purpose

CERA Medical will provide a public website and a controlled service-enquiry platform. Visitors can learn about approved services, read approved health and company content, submit an enquiry, receive confirmation, create an account, and view a safe status timeline. Authorized CERA staff can manage content and enquiries without direct database access. The release is designed for the current baseline of no more than about 100 active customers and about 5,000 monthly visits, with room to scale vertically before introducing distributed infrastructure.

1.2 Launch definition

- The public site, CMS, service catalogue, enquiry workflow, customer account, staff workflow, Zoho, Resend, Authentik, R2, monitoring, backups, staging, and production all operate end to end.
- Every protected route enforces role and record ownership on the server. UI hiding alone is never treated as authorization.
- No online payment, checkout, appointment booking, diagnosis, prescription, clinical record, or patient-document upload is included in this release.
- CERA owns every production vendor account and grants developers only the access needed for delivery and support.
- Production is released from a tagged and approved commit with a tested rollback target.

1.3 Product outcomes

Outcome	Launch measure
Visitors understand CERA services	All approved services have a discoverable detail page, clear CTA, correct metadata, and accessible mobile presentation.
Enquiries are captured reliably	Validated submissions create one enquiry, one reference number, one CRM record, and one customer receipt despite safe retries.
Customers can follow progress	Verified customers see only their own enquiries and customer-safe status history.
Staff can operate without developers	Content and operations owners complete the agreed UAT scripts using role-controlled interfaces.
The team can release safely	CI, staging, production approval, backups, monitoring, rollback, and ownership handover are evidenced.

2 Users roles and approval ownership

Role	Allowed responsibilities	Prohibited or restricted actions
Public visitor	Browse published content and services; submit a consented enquiry.	Cannot read enquiries, drafts, staff routes, or private operational data.
Customer	Manage own profile and view own customer-safe enquiry status after identity verification.	Cannot see internal notes, staff states, other customers, or CRM details.
Content Editor	Create and edit drafts, upload approved media, and request review.	Cannot publish medical claims, service copy, pricing, or policies.
Content and Clinical Approver	Review preview, publish, unpublish, and restore approved content.	Cannot change production infrastructure or release code.
Operations Support	View assigned enquiries, update allowed states, add private notes, and contact customers under CERA policy.	Cannot administer identities, secrets, deployments, or content publication.
Administrator	Manage users, role bindings, approved system settings, and audit access.	Uses MFA; cannot bypass release evidence or expose secrets.
CERA Product Owner	Approve product scope, staging UAT, launch content, and accepted follow-up defects.	Does not approve their own technical deployment if they initiated it.
Technical Release Approver	Confirm technical readiness, backup status, migration safety, rollback, and monitoring.	Cannot replace CERA product approval.

3 Product scope

3.1 Included capabilities

- Public pages, service catalogue, article publishing, search and filters, SEO metadata, sitemap, redirects, responsive behavior, and WCAG 2.2 AA target.
- Payload draft, preview, version, approval, publish, unpublish, media, and restore workflow.
- Vendure service catalogue used for categories, service records, display pricing, availability text, slugs, and enquiry eligibility. Transactional checkout remains disabled.
- Secure enquiry capture, duplicate protection, consent timestamp, customer receipt, staff alert, CRM synchronization, assignment, status transitions, private notes, and audit history.
- Authentik OIDC accounts for customers and staff, administrator MFA, server-side role enforcement, and environment-specific clients.
- Cloudflare DNS and edge controls, R2 media, Caddy HTTPS and reverse proxy, Docker Compose, PostgreSQL, Valkey, GlitchTip, backups, CI/CD, staging, production, release tags, rollback, and hotfixes.

3.2 Excluded capabilities

- Online payments, refunds, checkout, invoicing, subscriptions, stored payment data, and financial reconciliation.
- Appointment scheduling, telemedicine, electronic medical records, diagnoses, prescriptions, laboratory data, and clinical decision support.
- Customer uploads of medical records, identity documents, or health documents.
- Native mobile applications, multi-language content, marketing automation campaigns, and advanced analytics unless added through a separately approved change.
- Formal legal, medical-device, privacy, or healthcare compliance certification. CERA must obtain applicable legal and clinical review before launch.

3.3 Fixed operating decisions

Decision	Baseline
Repository ownership	CERA-owned GitHub organization and private repository. Stack & Scale and Dev 3 receive administrator access during setup; CERA retains at least two owner accounts.
Branch mapping	develop deploys to staging; main represents production; features start from develop; releases start from develop; hotfixes start from main.
Content approval	One named CERA Content and Clinical Approver must approve service, health, pricing, policy, and blog content before publication.
Release approval	Dev 3 records technical approval and the CERA Product Owner records UAT approval before the production environment releases secrets.
Data boundary	Collect name, email, phone, selected service, message, consent, source, ownership, status, and audit metadata. Do not collect clinical history or health documents.
Vendor ownership	CERA owns and pays GitHub, Hetzner, Cloudflare, R2, Zoho, Resend, domain, and backup accounts.
Jurisdiction	Swabi, Khyber Pakhtunkhwa, Pakistan, subject to final contract and legal review.

4 End to end user journeys

4.1 Visitor to enquiry

1. The visitor opens a published page or service and sees content approved by CERA.
2. The visitor selects a service and opens the enquiry form with the service already identified.
3. The form explains the data use, requires consent, validates fields, and applies anti-abuse controls.
4. The API creates one enquiry and audit event, then queues Zoho and Resend work without delaying the response.
5. The visitor receives a reference number and confirmation email. Staff receive a notification and the record appears in the operations queue.

4.2 Customer account and status

1. The customer signs in or creates an account through Authentik and verifies control of the enquiry email address.
2. A short-lived, single-use claim token links eligible earlier enquiries to that identity.
3. The dashboard requests records by authenticated subject ID and the API applies ownership filters before returning data.
4. The customer sees a simplified status timeline and never receives internal notes, CRM fields, or staff-only states.

4.3 Content publication

1. A Content Editor creates or updates a draft and attaches approved media.
2. The editor previews the exact staging presentation and requests approval.
3. The Content and Clinical Approver checks wording, claims, pricing, links, and metadata, then publishes or returns the draft.
4. Payload stores version history so an approver can restore a previous version. Public APIs return published content only.

4.4 Staff enquiry handling

1. Operations Support filters the queue, opens an enquiry, assigns an owner, and records a private note.
2. The API validates each status transition and writes an audit event.
3. The customer-safe projection updates the dashboard. The Zoho worker updates the mapped record idempotently.
4. Completion or closure records the final state, actor, and timestamp. Retention and deletion follow the approved CERA policy.

5 Delivery phases

Phase	Days	Purpose	Exit condition
Phase 0 Readiness	Before Day 1	Accounts, content owners, domains, vendor access, repository, scope, and test data are available.	No external-access blocker remains; CERA names Product Owner and Content and Clinical Approver.
Phase 1 Foundation	Days 1-2	Repository, local stack, contracts, catalogue baseline, UI foundation, CI, environments, and rulesets.	All developers can work from contracts and protected PRs; staging prerequisites are ready.
Phase 2 Core product	Days 3-6	CMS, public pages, services, enquiry, CRM, email, identity, storage, staging, customer ownership, and backups.	A staging visitor can browse, enquire, receive email, create an account, and see an owned record.
Phase 3 Operations	Day 7	Customer dashboard, staff operations, private notes, audit history, and observability.	CERA can process a seeded enquiry and diagnose a deliberate test error.
Phase 4 Verification	Days 8-9	Automated tests, security review, accessibility, recovery rehearsal, content approval, and UAT fixes.	No unresolved critical or high finding and all launch-blocking UAT defects are closed.
Phase 5 Release	Day 10	Approved release branch, tag, production deployment, smoke tests, monitoring, rollback, and handover.	v1.0.0 is live with release evidence, backup status, rollback reference, and named owners.

6 Feature catalogue and acceptance criteria

Each row is a deliverable unit and must be represented by one or more GitHub Issues. The owner may split implementation across focused pull requests, but the acceptance statement remains the feature's release condition.

ID	Phase	Feature	Acceptance evidence	Owner	Day
FND-001	Phase 1	Repository and monorepo foundation	Fresh clone installs and all workspace commands run; no secret or generated artifact is tracked.	Dev 3	Day 1
FND-002	Phase 1	Local service stack	A new developer follows README steps and reaches all local health endpoints within 30 minutes.	Dev 3	Day 1
FND-003	Phase 1	Shared contracts and fixtures	Web, API, and tests import the same contracts; contract tests reject incompatible changes.	Dev 2	Day 1
FND-004	Phase 1	Continuous integration checks	Required checks report independently and a failing required check blocks merge.	Dev 3	Day 2
FND-005	Phase 1	Environment contract	The repository contains .env.example files with placeholders; real values exist only in approved secret stores.	Dev 3	Day 2
CMS-101	Phase 2	Payload content models	Editors can create and preview each content type in staging; public APIs return only published records.	Dev 1	Day 3
CMS-102	Phase 2	Editorial approval workflow	An editor cannot publish; an approver can preview, publish, and restore a previous version with an audit trail.	Dev 1	Day 3
CAT-201	Phase 2	Vendure service catalogue	Seeded services are available through a stable read API and unpublished services never appear publicly.	Dev 2	Day 2
WEB-301	Phase 2	Public website foundation	Core pages work on current mobile and desktop browsers, keyboard navigation is complete, and automated accessibility checks pass.	Dev 1	Day 1-2
WEB-302	Phase 2	Public content pages	Each route has approved copy, metadata, canonical URL, sharing metadata, and a useful empty or error state.	Dev 1	Day 4-5
WEB-303	Phase 2	Search and discoverability	Search and filters return deterministic results; draft content is excluded; sitemap includes only indexable routes.	Dev 1	Day 5
ENQ-401	Phase 2	Secure enquiry submission	Valid enquiries are stored once; invalid requests return field-level errors; duplicate submissions do not create duplicate records.	Dev 2	Day 3
ENQ-402	Phase 2	Enquiry experience	A keyboard-only user can submit successfully; confirmation displays the reference number; errors preserve safe form fields.	Dev 1	Day 6
ENQ-403	Phase 2	Enquiry status model	Invalid transitions are rejected; customers never receive internal notes or staff-only status values.	Dev 2	Day 6-7
AUTH-501	Phase 3	Authentik identity provider	Staging login, logout, token validation, session expiry, and admin MFA are verified with test accounts.	Dev 3	Day 3
AUTH-502	Phase 3	Role based access control	Automated authorization tests prove each role can only access approved routes, records, and actions.	Dev 3	Day 3-4
CUS-601	Phase 3	Customer account and enquiry claiming	One account cannot claim another email address's enquiry; expired or reused claim tokens fail safely.	Dev 2	Day 6
CUS-602	Phase 3	Customer dashboard	A customer sees only their own records; empty, loading, expired-session, and API-failure states are tested.	Dev 1	Day 7
OPS-701	Phase 3	Operations enquiry queue	Operations users can process seeded enquiries without database access; unauthorized roles receive no data.	Dev 2	Day 7
OPS-702	Phase 3	Internal notes and audit history	Customer APIs and exports contain no internal notes; audit events show who changed what and when.	Dev 2	Day 7
INT-801	Phase 3	Zoho CRM synchronization	A retry updates the same CRM record; failures enter a retry queue and remain visible without losing the enquiry.	Dev 2	Day 4

ID	Phase	Feature	Acceptance evidence	Owner	Day
INT-802	Phase 3	Resend transactional email	A repeated job sends one email; verified webhook events update delivery status; bounce and complaint events are recorded.	Dev 2	Day 5
INT-803	Phase 3	Cloudflare R2 media storage	Upload, replace, delete, render, cache, file-type, and file-size rules work in staging; bucket credentials are not exposed to clients.	Dev 3	Day 4
INF-901	Phase 3	Caddy and Cloudflare edge	Only ports 80 and 443 are public; database and cache ports are private; health checks and forwarded headers work.	Dev 3	Day 4
INF-902	Phase 3	Staging deployment	A successful merge updates staging without manual file copying; a failed health check restores the previous image.	Dev 3	Day 5
INF-903	Phase 4	Production deployment	Production secrets are unavailable before approval; deployment records version, approver, image digest, migration, and rollback target.	Dev 3	Day 10
INF-904	Phase 3	Backup and restore	The team restores a staging database from backup and records recovery time and integrity checks before launch.	Dev 3	Day 6
OBS-1001	Phase 3	Monitoring and error reporting	A deliberate staging error reaches the correct project without secrets or personal message content.	Dev 3	Day 7
QA-1101	Phase 4	Automated release test suite	Required checks pass from a clean clone; failures are reproducible locally; test fixtures contain no real customer data.	All	Day 8
QA-1102	Phase 4	Accessibility and performance verification	Automated checks pass and manual keyboard, focus, screen-reader smoke, mobile, and slow-network checks are recorded.	Dev 1	Day 8
REL-1101	Phase 4	CERA user acceptance testing	Every launch-blocking defect is closed; lower-priority defects have an owner and accepted follow-up date; approvals are recorded.	All	Day 9
REL-1102	Phase 4	Release and rollback	The release checklist, change log, rollback reference, and ownership handover are complete.	Dev 3	Day 10
DOC-1201	Phase 4	Operations handover	A named CERA owner can execute routine content and enquiry tasks without developer assistance.	All	Day 10

7 Functional requirements by module

Public website

- Render only published CMS content and active services.
- Provide responsive navigation, clear enquiry calls to action, canonical metadata, sitemap, robots policy, redirects, and meaningful error states.
- Meet the WCAG 2.2 AA target through semantic HTML, keyboard operation, visible focus, labels, contrast, status announcements, and manual checks. [R21]

Payload CMS

- Use drafts and versions for pages, posts, policies, service presentation, navigation, media, SEO, and redirects. [R8]
- Restrict publishing to the Content and Clinical Approver. Editors can draft and preview.
- Use the official S3 storage adapter for R2 in the Node.js deployment and keep local storage disabled outside local development. [R7]

Vendure catalogue

- Use Vendure as the service catalogue and admin interface; do not expose checkout in this release.
- Run the server and worker separately. Use a persistent queue strategy; an in-memory queue is not suitable for production. [R9]
- Treat service IDs and slugs as stable contracts referenced by enquiries and CMS presentation records.

Enquiry and operations

- Validate on the server, normalize fields, rate limit, record consent, prevent duplicate processing, and create an immutable reference.
- Maintain internal and customer-safe status models with explicit transitions.
- Keep internal notes staff-only and record each change with actor and timestamp.

Identity

- Use Authentik as the OIDC provider with separate clients and redirect URLs for local, staging, and production. [R11]
- Require MFA for administrators and release approvers; use short sessions for privileged interfaces.
- Keep discovery, JWKS, token, and callback routes reachable for server-to-server authentication and configure trusted proxy behavior. [R10][R11]

Zoho

- Use Zoho CRM V8 OAuth and the narrowest module scopes required. Access and refresh tokens never reach browser code. [R15][R16]
- Upsert by external enquiry ID and reconcile retries without duplicates.
- Store response IDs and errors so staff can identify and retry failed synchronization.

Resend

- Use a CERA-owned verified sending domain and separate test/production API keys.
- Use an idempotency key based on event type and enquiry ID; Resend retains keys for its documented retry window. [R17]
- Verify webhook signatures before accepting delivery, bounce, or complaint events.

Cloudflare R2

- Use separate staging and production buckets and least-privilege tokens.
- Public editorial media may use approved public delivery; any future private asset must use access control and short-lived server-generated access.
- Treat presigned URLs as bearer tokens and keep their scope and expiry minimal. [R12]

8 Data model and interface contracts

Entity	Required fields	Ownership and exposure
Service	id, slug, category, title, summary, description, displayPrice, availabilityText, enquiryEnabled, mediaId, status, timestamps	Vendure owns commercial/service data. Public API returns active fields only.
ContentDocument	id, type, slug, title, blocks/body, SEO, media references, status, author, approver, timestamps	Payload owns content. Public reads published records; authenticated preview reads drafts.
Enquiry	id, reference, customerSubjectId?, name, email, phone, serviceId, message, consentAt, source, internalStatus, ownerId, timestamps	Staff can read assigned/authorized records. Customer projection removes internal fields.

Entity	Required fields	Ownership and exposure
EnquiryStatusEvent	id, enquiryId, previousStatus, newStatus, customerStatus, actorSubjectId, reason, createdAt	Immutable audit event. Customer API returns approved status label and timestamp only.
InternalNote	id, enquiryId, authorSubjectId, body, createdAt, editedAt?	Staff-only. Never included in customer, public, or Zoho payloads.
IntegrationDelivery	id, enquiryId, provider, eventType, idempotencyKey, externalId, attempt, status, responseCode, errorClass, timestamps	Operations and administrators only. Secrets and full tokens are never stored.
CustomerProfile	subjectId, email, displayName, phone?, emailVerifiedAt, createdAt, updatedAt	Identity keyed by Authentik subject. Customer reads and updates approved own fields.
AuditEvent	id, actorSubjectId, action, targetType, targetId, safeDiff, requestId, createdAt	Append-only administrative evidence with personal data minimized.

8.1 Contract rules

- Contracts live in packages/contracts and are imported by producers, consumers, mocks, and tests.
- Every endpoint returns a request ID and a stable error envelope with code, message, field errors, and retryability. Stack traces remain server-side.
- Breaking contract changes require a new issue, Dev 1 and Dev 2 agreement, migration notes, and updated fixtures before implementation.
- External calls use an outbox or durable job record so successful local writes are not lost when Zoho or Resend is unavailable.

9 Technical architecture

Internet

- > Cloudflare DNS and edge controls
- > Caddy HTTPS and reverse proxy
 - > Next.js public and customer web
 - > Payload CMS and admin
 - > Vendure API and admin
 - > Authentik identity
 - > GlitchTip monitoring

Private Docker networks

- > PostgreSQL with separate databases and least-privilege roles
- > Valkey with separated namespaces or instances
- > Vendure worker and integration workers

External services

- > Cloudflare R2 media
- > Zoho CRM
- > Resend transactional email
- > GitHub Actions and GHCR

9.1 Deployment topology

Environment	Branch or trigger	Infrastructure	Data and access
Local	Developer feature branch	Local applications plus Docker Compose dependencies	Synthetic fixtures; local secrets only; no production vendor credentials.
Staging	develop after required checks	Hetzner CX33 or equivalent; same container topology as production	Separate databases, buckets, OAuth clients, Zoho sandbox/test configuration, and Resend test domain.
Production	Approved v* tag from main	Hetzner CX43 or equivalent; Caddy; immutable images from GHCR	Production-only secrets; CERA data; manual approval; backups, monitoring, and deletion protection.

The selected server sizes are procurement assumptions, not performance guarantees. The team must record CPU, memory, disk, database connections, latency, and error rate during staging tests and resize before launch if the agreed budgets fail.

9.2 Repository structure

```

cera-platform/
  apps/
    web/           Next.js public and customer application
    cms/           Payload CMS and admin
    commerce/      Vendure server, worker, and admin
  packages/
    contracts/     shared schemas, API types, fixtures
    ui/            shared accessible UI components
    config/        shared lint, TypeScript, and test config
    observability/ logging, tracing, GlitchTip helpers
  infra/
    compose/       base, local, staging, production overlays
    caddy/         Caddyfile and security headers
    scripts/       deploy, migrate, backup, restore, smoke test
  docs/
    adr/           architecture decisions
    runbooks/      deploy, rollback, backup, incident, access
    contracts/     human-readable API and data documentation
  .github/
    workflows/     ci, staging, release, security
    ISSUE_TEMPLATE/
    pull_request_template.md
    CODEOWNERS
  .env.example
  compose.yaml
  pnpm-workspace.yaml
  README.md

```

10 Nonfunctional requirements

Area	Requirement and test
Availability	Target 99.5 percent monthly availability after launch, excluding approved maintenance. Health checks cover public web, APIs, workers, database connectivity, and disk.
Performance	Under the baseline traffic assumption, target p75 LCP at or below 2.5 seconds on representative mobile tests; API p95 below 800 ms excluding external-provider latency; no unbounded database queries.
Accessibility	Target WCAG 2.2 AA for all public and customer routes, verified with automated checks and manual keyboard, focus, label, status, contrast, zoom, and screen-reader smoke tests. [R21]
Security	Address OWASP Top 10 2025 risks, especially access control, misconfiguration, supply chain, authentication, integrity, logging, and exceptional conditions. [R22]
Privacy	Collect the minimum launch data, show purpose and consent, separate internal notes, prohibit clinical uploads, redact sensitive fields from logs, and obtain CERA legal approval for notice, retention, deletion, and vendor terms.
Recovery	Target RPO 24 hours and RTO 4 hours for the baseline launch. Daily encrypted database dumps and Hetzner backups are separate controls; Hetzner backups do not include attached volumes. [R18][R19]
Maintainability	Pinned tool versions, locked dependencies, typed contracts, reversible or backward-compatible migrations, structured logs, runbooks, and architecture decisions are required.
Observability	Every request has a request ID; errors include release and environment; alerts avoid enquiry messages, access tokens, passwords, and other unnecessary personal data.

11 GitHub repository and branch controls

11.1 Repository creation

1. CERA creates or confirms its GitHub organization and two owner accounts with MFA.
2. Stack & Scale or Dev 3 creates the private cera-platform repository under that organization, never under a developer's personal account.
3. Dev 3 adds the baseline files, opens the bootstrap pull request, and records the repository settings in the release runbook.
4. Developers receive write access. CERA Product Owner receives read access plus the access required for deployment review. Administrator access is limited and reviewed after handover.

11.2 Branch model

Branch	Starts from	Merges to	Purpose
main	Release or hotfix PR	Production	Production-ready history and version tags only.
develop	main at initialization	Staging	Daily integrated release candidate for staging.
feat/*, fix/*, test/*, ci/*, chore/*	develop	develop by PR	Short-lived single-purpose work linked to one issue.
release/vX.Y.Z	develop	main by PR; back to develop if changed	Release stabilization and approved release-only fixes.
hotfix/*	main	main and develop by PR	Critical production correction with focused tests and incident reference.

11.3 Rulesets

Target	Required controls
develop	Pull request required; one approval; code-owner approval for owned paths; dismiss stale approvals; require approval after latest push; resolve conversations; pass required checks; block deletion and force push; apply to administrators.
main	Pull request required; two approvals; code-owner approval; latest-push approval; resolved conversations; all release checks; only squash merge; signed release tag; block deletion and force push; apply to administrators.
Tags v*	Restrict creation and deletion to release maintainers; tag must point to a main commit that passed the release workflow.
Production environment	Allow only main or protected v* tags; require CERA and technical approval where the GitHub plan supports it; prevent self-review; release secrets only after approval. [R2]

GitHub rulesets can enforce reviews, code-owner approval, latest-push approval, conversation resolution, and allowed merge types. [R1] Availability of some environment protection features depends on repository visibility and GitHub plan. If the selected plan cannot enforce the approval electronically, the workflow must verify a signed release-approval issue before deployment.

12 Daily Git workflow and exact commands

12.1 One-time clone and configuration

```
git clone <CERA_REPOSITORY_URL>
cd cera-platform
git config user.name "Your Full Name"
git config user.email "your-cera-email@example.com"
git config pull.ff only
git config fetch.prune true
corepack enable
pnpm install --frozen-lockfile
cp .env.example .env.local
docker compose up -d postgres valkey
pnpm dev
```

12.2 Start every morning

Do not update branches while files are uncommitted. First commit finished work or stash temporary work. Then refresh develop and merge it into the active feature branch.

```
git status
# If temporary work must be saved:
git stash push -u -m "WIP before morning sync"

git fetch --prune origin
git switch develop
git pull --ff-only origin develop
git switch <your-feature-branch>
git merge --no-edit develop

# If work was stashed:
git stash pop

pnpm install --frozen-lockfile
pnpm lint
pnpm typecheck
pnpm test
```

12.3 Start a new feature

```
git fetch --prune origin
git switch develop
git pull --ff-only origin develop
git switch -c feat/cera-123-short-feature-name
```

12.4 Commit and push

```
git status
git diff
git add -p
git diff --cached
git commit -m "feat(enquiry): validate consent and service id"
git push -u origin $(git branch --show-current)
```

12.5 Open and update a pull request

```
gh pr create \
  --base develop \
  --head $(git branch --show-current) \
  --title "feat(enquiry): add validated submission flow" \
  --body-file .github/pull_request_template.md

gh pr checks --watch
gh pr view --web

# Later changes use normal commits and pushes:
git add -p
git commit -m "fix(enquiry): preserve fields after retryable error"
git push
```

12.6 Sync an active feature branch

```
git status
git fetch --prune origin
git switch develop
git pull --ff-only origin develop
git switch <your-feature-branch>
git merge --no-edit develop
pnpm lint && pnpm typecheck && pnpm test
git push
```

12.7 Resolve a merge conflict

```
git status
# Edit each conflicted file and remove conflict markers.
git diff --check
git add <resolved-file-1> <resolved-file-2>
git commit
pnpm lint && pnpm typecheck && pnpm test
git push

# If the merge should be cancelled before committing:
git merge --abort
```

12.8 Review and merge

```
gh pr checkout <PR_NUMBER>
pnpm install --frozen-lockfile
pnpm lint && pnpm typecheck && pnpm test && pnpm build
gh pr review <PR_NUMBER> --approve

# Authorized maintainer after approvals and green checks:
gh pr merge <PR_NUMBER> --squash --delete-branch
```

12.9 After merge

```
git switch develop
git pull --ff-only origin develop
git branch -d <merged-feature-branch>
git fetch --prune origin
```

Developers must never push directly to main or develop, force-push a shared branch, commit .env files, copy production data into local machines, or bypass a required check. This workflow uses merge updates for beginner safety; no force push is needed.

13 Issues pull requests and review evidence

13.1 Issue workflow

- Create one GitHub Milestone for v1.0.0 and one Epic issue per delivery phase.
- Create one issue for each feature ID with user value, requirements, acceptance criteria, dependencies, owner, reviewer, test data, and launch impact.
- Use statuses Backlog, Ready, In progress, In review, Staging, UAT, Done and labels area:*, type:*, priority:*, risk:*, and dev:*
- A developer moves an issue to In progress only after dependencies and contracts are available. A merged PR moves it to Staging; acceptance evidence and approval move it to Done.

13.2 Pull request requirements

- Title uses Conventional Commits: type(scope): specific result. The body links the issue and feature ID.
- Explain what changed, why, main implementation choices, tests run, screenshots for UI, migration impact, secrets or environment changes, and rollback behavior.
- Keep a PR focused. If the diff becomes difficult to review, split it by contract, backend, frontend, tests, or infrastructure while keeping the same issue relationship.
- The author completes a self-review before requesting review. Reviewers check behavior, access control, data boundaries, error paths, tests, migration safety, observability, and documentation.

13.3 CODEOWNERS baseline

/apps/web/	@cera/dev-1 @cera/dev-3
/apps/cms/	@cera/dev-1 @cera/dev-3
/apps/commerce/	@cera/dev-2 @cera/dev-3
/packages/contracts/	@cera/dev-1 @cera/dev-2
/infra/	@cera/dev-3 @cera/dev-2
/.github/workflows/	@cera/dev-3
/**/*.migrations/	@cera/dev-2 @cera/dev-3
/docs/runbooks/	@cera/dev-3

14 Continuous integration and deployment

Trigger	Pipeline	Result
Pull request to develop	Install locked dependencies; format; lint; type check; unit; contract; integration; build; migration check; dependency review; secret scan.	Review evidence only. No production credentials and no production deployment.
Merge to develop	Build once; tag image with commit SHA; publish to GHCR; migrate staging once; deploy Compose overlay; health and smoke tests; record deployment.	Staging contains the exact merged SHA or is automatically restored to the previous healthy image.
Release PR to main	Run complete test suite, security checks, migration dry run, release notes, backup confirmation, and UAT approval verification.	Main receives only an approved release candidate.
Approved v* tag	Unlock production environment secrets; pull the previously built digest; backup; migrate; deploy; smoke test; annotate GlitchTip release; monitor.	Immutable production release with approvers, image digest, migration, and rollback target recorded.

Trigger	Pipeline	Result
Hotfix PR	Focused tests plus full critical-path suite; incident link; production approval; deploy patch tag; merge correction back to develop.	Production corrected without losing the fix from the next release.

Third-party actions must be pinned to reviewed full commit SHAs because GitHub identifies that as the immutable reference. Workflow permissions must default to read and be increased only per job. [R3] Images are published to GHCR with explicit packages permissions and tagged with the source SHA and release version. [R4]

14.1 Deployment command pattern

```
docker compose -f compose.yaml -f infra/compose/compose.production.yaml pull
docker compose -f compose.yaml -f infra/compose/compose.production.yaml run --rm migrate
docker compose -f compose.yaml -f infra/compose/compose.production.yaml up -d --remove-orphans
./infra/scripts/health-check.sh
./infra/scripts/smoke-test.sh
```

Production uses a dedicated Compose overlay with immutable images, restart policies, production environment values, no source-code bind mounts, and only required host ports, consistent with Docker's production guidance. [R20] Next.js and Payload use standalone production output in container images. [R5][R6]

15 Secrets and environment handling

Location	Allowed	Prohibited
Repository	.env.example names, safe defaults, schema, documentation	Passwords, API keys, tokens, private keys, customer data, production dumps
Developer machine	.env.local with local or sandbox values; ignored files; password manager reference	Shared production credentials, copied production database, secrets in screenshots or chat
GitHub staging environment	Staging-only deployment key, database URL, OAuth clients, R2, Resend, Zoho test values	Production values or administrator/root credentials
GitHub production environment	Production-only least-privilege values released after approval	Secrets accessible to PR workflows or untrusted branches
Server	Root-readable or service-readable secret file outside repository; Docker secrets where practical	Secrets baked into images, command history, world-readable files, application logs

- Rotate any exposed secret immediately and after the developer handover. Record owner, purpose, environment, created date, rotation date, and revocation procedure without recording the value.
- Use separate credentials for local, staging, and production. Never make a staging key valid against production data.
- Deploy through a non-root server account with access limited to the deployment directory and required Docker operations.

16 Test strategy and launch evidence

Layer	Required coverage	Owner
Unit	Validation, status mapping, permissions helpers, idempotency keys, content transforms, and utility functions.	Feature owner
Contract	Shared schemas, mock fixtures, API request and response compatibility, error envelopes, and safe projections.	Dev 1 and Dev 2
Integration	PostgreSQL migrations, repositories, queues, Authentik token validation, Zoho and Resend adapters, R2 behavior, and audit events.	Dev 2 and Dev 3
Authorization	Role and record ownership matrix with negative tests for every protected action.	Dev 2 and Dev 3
Browser	Published browse, service discovery, enquiry, receipt, login, claim, dashboard, staff processing, logout, errors, mobile, and keyboard.	Dev 1
Operations	Backup and restore, deployment rollback, monitoring alert, disk pressure, provider outage, and worker restart.	Dev 3
UAT	Content approval, service management, visitor flow, customer flow, operations flow, CRM, email, and release approval.	CERA Product Owner

16.1 Test data rules

- Use synthetic people, safe example phone numbers, non-sensitive messages, test services, and vendor sandbox or test records.
- Never copy production customer or enquiry data to local or staging. If a defect requires production evidence, minimize and redact it before controlled access.
- Version fixtures with contracts so each developer can work before another service is available.

17 Backup monitoring and incident requirements

- Run daily encrypted PostgreSQL dumps, retain an agreed rotation, copy them off the production host, and test restoration before launch and monthly after launch. PostgreSQL documents SQL dumps, file-system backup, and continuous archiving as distinct approaches. [R19]
- Enable Hetzner daily backups and deletion protection. Attached volumes require a separate backup method because Hetzner server backups and snapshots do not include them. [R18]
- Self-host GlitchTip behind Caddy, keep it private to authorized users, set retention appropriate to disk capacity, and send only minimized event context. GlitchTip supports Docker Compose, PostgreSQL, and optional Valkey for the baseline deployment. [R14]
- Caddy terminates HTTPS and renews public certificates automatically when DNS, ports, hostname, and persistent storage are configured. [R13]
- Classify incidents as Sev 1 unavailable or data exposure, Sev 2 critical function impaired, Sev 3 limited defect, and Sev 4 minor issue. Sev 1 triggers rollback or containment before further feature work.

18 Integrated ten day delivery schedule

Day	Developer 01	Developer 02	Developer 03	End of day gate
1	feat/cera-101-web-foundation WEB-301 Create the Next.js application shell, route map, base layout, navigation, footer, error boundaries, loading states, and shared UI package.	chore/cera-201-contracts-data FND-003 Define packages/contracts, PostgreSQL schemas, migrations, seed fixtures, status enums, error envelopes, and API ownership rules.	chore/cera-301-platform-bootstrap FND-001, FND-002 Create the CERA-owned repository structure, local Compose stack, health endpoints, private networks, least-privilege users, and base server runbook.	Contracts and local stack available
2	feat/cera-102-design-system WEB-301, QA-1102 Complete responsive components, forms, cards, tables, empty states, toast/status messages, and initial accessibility tests.	feat/cera-202-service-catalogue CAT-201 Configure Vendure service categories, service records, slugs, display pricing, availability text, media references, and read API. Disable checkout routes.	ci/cera-302-quality-gates FND-004, FND-005 Configure GitHub rulesets, CODEOWNERS, Actions permissions, pinned actions, CI jobs, environment contracts, and staging/production GitHub environments.	Protected PR workflow and catalogue baseline ready
3	feat/cera-103-payload-content CMS-101, CMS-102 Implement Payload collections and globals for pages, navigation, services presentation, blog, policies, media, SEO, drafts, versions, and publish permissions.	feat/cera-203-enquiry-api ENQ-401 Build enquiry validation, consent timestamp, reference generation, rate limiting, duplicate protection, persistence, and audit event creation.	feat/cera-303-authentik-oidc AUTH-501, AUTH-502 Deploy Authentik for staging, configure OIDC clients, roles, admin MFA, recovery email, trusted proxies, and test accounts.	CMS, enquiry, and OIDC foundations demonstrated
4	feat/cera-104-service-pages WEB-302 Build service listing and detail routes from the shared catalogue contract, with filters, CTA, metadata, and safe missing-service handling.	feat/cera-204-zoho-sync INT-801 Implement least-privilege Zoho OAuth, external enquiry ID mapping, Lead or Contact upsert, retry policy, dead-letter visibility, and reconciliation command.	feat/cera-304-edge-r2 INT-803, INF-901 Configure Cloudflare DNS, R2 buckets and tokens, Caddy HTTPS/reverse proxy, security headers, upload limits, and private service networks.	Services, Zoho, edge, and R2 integrated
5	feat/cera-105-blog-seo WEB-302, WEB-303 Build blog listing and article routes, sitemap, robots policy, canonical URLs, social metadata, media rendering, and content preview.	feat/cera-205-resend-notifications INT-802 Send customer and staff emails with idempotency keys; verify delivery webhooks; record sent, delivered, bounced, and complained events.	feat/cera-305-staging-deploy INF-902 Build and publish immutable GHCR images, deploy develop to CX33 staging, run controlled migrations and health checks, and record image digests.	Blog, email, and automatic staging deployment integrated
6	feat/cera-106-enquiry-experience ENQ-402 Build the validated enquiry form, consent copy, anti-abuse UI, success reference, retry states, and no-data-loss error behavior.	feat/cera-206-customer-ownership CUS-601, ENQ-403 Implement customer-safe status mapping, enquiry claim tokens, ownership checks, profile APIs, and status detail APIs.	feat/cera-306-backup-restore INF-904 Enable Hetzner backups, encrypted PostgreSQL dumps to off-server storage, retention, restore scripts, and configuration recovery documentation.	Customer ownership and tested restore complete

Day	Developer 01	Developer 02	Developer 03	End of day gate
7	feat/cera-107-customer-dashboard CUS-602 Implement Authentik sign-in entry, protected dashboard, profile summary, enquiry list, timeline, session-expiry handling, and empty states.	feat/cera-207-operations-api OPS-701, OPS-702 Implement staff queue filters, assignment, valid transitions, internal notes, audit history, and safe customer/CRM projections.	feat/cera-307-glitchtip-observability OBS-1001 Deploy self-hosted GlitchTip, configure release-tagged projects, structured logs, uptime checks, disk and backup alerts, and alert routing.	Customer, staff, and monitoring flows complete
8	test/cera-108-web-e2e QA-1101, QA-1102 Write browser tests for browse, enquiry, confirmation, login, claim, status, and logout. Complete accessibility, mobile, and slow-network checks.	test/cera-208-integration-suite QA-1101 Complete migration, contract, authorization, Zoho retry, Resend webhook, backup-restore data, and queue recovery tests.	test/cera-308-security-release QA-1101 Run access, secret, dependency, container, firewall, headers, rate-limit, session, backup, and disaster-recovery checks.	Release test and security evidence complete
9	fix/cera-109-uat-frontend REL-1101 Resolve CERA UAT defects, apply only approved content corrections, and freeze nonessential visual changes.	fix/cera-209-uat-integrations REL-1101 Resolve UAT data and integration defects, reconcile staging records, and verify migration backward compatibility.	fix/cera-309-uat-release REL-1101 Support UAT fixes, freeze platform changes, prepare release branch, validate production configuration, and confirm rollback owner and command.	CERA UAT and launch approvals ready
10	release/cera-110-web-v1 DOC-1201 Run production smoke tests for pages, CMS publishing, services, enquiry, login, dashboard, responsive layout, and analytics-free privacy behavior.	release/cera-210-api-v1 DOC-1201 Run production API smoke tests, verify worker queues, Zoho and Resend delivery, and hand over field, status, and recovery references.	release/cera-310-production-v1 INF-903, REL-1102, DOC-1201 Approve the production workflow, deploy v1.0.0 to CX43, run smoke tests, monitor 24 hours, and transfer access and runbooks.	v1.0.0 deployed, monitored, and handed over

19 Daily team operating rhythm

Time	Activity	Required output
09:15	Each developer runs the morning Git sync and checks assigned issues and dependencies.	Clean branch synchronized with develop or a named blocker.
09:30	Fifteen-minute stand-up: yesterday's evidence, today's feature, blocker, risky files, and required handoff.	Issue status and handoff owner updated.
10:00-13:00	Focused implementation on one named branch. Contracts are changed only after affected owners agree.	Small commits with local checks.
13:00	Integration checkpoint. Dev 1 verifies mocks/contracts, Dev 2 verifies APIs/data, Dev 3 verifies environment/runtime.	Contract or environment mismatch resolved early.
14:00-16:30	Complete feature slice, tests, documentation, self-review, push, and PR.	Reviewable PR with evidence.
16:30	Cross-review. High-risk auth, migration, workflow, secret, or deployment work receives two reviewers.	Approval, requested changes, or documented blocker.
17:30	Merge ready work, deploy develop to staging, run smoke checks, update issue and next-day handoff.	Staging evidence and accurate board status.

If a feature misses its daily gate, the owner must split the remaining work into a new issue, keep incomplete behavior disabled, and inform every dependent owner. Security, authorization, migration safety, backup, and release controls are never removed to recover schedule.

20 Release rollback and hotfix procedure

20.1 Release

```
git fetch --prune origin
git switch develop
git pull --ff-only origin develop
git switch -c release/v1.0.0
git push -u origin release/v1.0.0
gh pr create --base main --head release/v1.0.0 --title "release: v1.0.0"
# After approvals, required checks, and merge:
git switch main
git pull --ff-only origin main
git tag -a v1.0.0 -m "CERA Medical production release v1.0.0"
git push origin v1.0.0
```

20.2 Rollback

1. Stop the deployment if migrations, health checks, smoke tests, or authorization checks fail.
2. Restore the previously recorded image digest. Use a forward database correction unless the approved migration explicitly supports safe rollback.
3. Verify web, APIs, workers, login, enquiry, database, Zoho queue, Resend queue, and monitoring after rollback.
4. Record timeline, symptoms, impact, decision, commands, image digest, database action, and follow-up issue.

20.3 Hotfix

```
git fetch --prune origin
git switch main
git pull --ff-only origin main
git switch -c hotfix/cera-999-critical-description
# Implement focused change and tests.
git add -p
git commit -m "fix(scope): correct critical production defect"
git push -u origin hotfix/cera-999-critical-description
gh pr create --base main --head hotfix/cera-999-critical-description --title "fix(scope):
critical production defect"
# After production release, open a second PR from the hotfix branch to develop.
```

21 Definition of done and launch checklist

- The issue requirements and acceptance criteria are satisfied with linked automated or manual evidence.
- The branch contains no secret, production data, generated dependency directory, or unrelated change.
- Contracts, tests, migrations, documentation, logs, metrics, error behavior, and rollback notes are updated as applicable.
- Required reviewers approved the latest push, all conversations are resolved, and every required check is green.
- The merged commit is running in staging and the feature owner completed the staging acceptance steps.
- CERA approved launch content and UAT. Dev 3 approved technical readiness. The production environment recorded both approvals.
- The production release has a version tag, image digest, deployment record, smoke evidence, backup status, monitoring release, and rollback target.

22 CERA prerequisites before Day 1

Owner	Required item	Deadline
CERA	GitHub organization and private repository authority; two owner accounts with MFA	Before repository setup
CERA	Named Product Owner and Content and Clinical Approver with GitHub/email access	Before Day 1
CERA	Domain and Cloudflare access; approved subdomains for web, admin, auth, status, and monitoring	Before Day 1
CERA	Hetzner projects for CX33 staging and CX43 production or approved equivalents; SSH key owners	Before Day 1
CERA	R2, Resend, Zoho, and backup accounts owned by CERA; sandbox or test values where available	Before Day 2
CERA	Approved logo, brand assets, service list, service copy, display pricing, contact details, privacy and terms drafts, and launch blog content	Before Day 3
CERA	Written approval of data fields, consent text, retention, customer-safe statuses, Zoho field mapping, and email wording	Before Day 4
Stack & Scale	Repo bootstrap, issue import, rulesets, local template, and developer onboarding	Day 1

23 Risks and controls

Risk	Control	Owner
Ten-day schedule	Freeze scope, use fixed contracts and mocks, integrate daily, cut optional polish rather than safety or testing, and escalate blockers the same day.	Product Owner and Dev 3
Vendor access or approval delay	Complete the prerequisite register before Day 1 and use sandbox adapters until owned credentials arrive.	CERA
Overlapping files and conflicts	Path ownership, short branches, morning merge from develop, midday integration checkpoint, and small PRs.	All developers
Medical or personal data exposure	Exclude clinical data, minimize fields, enforce authorization server-side, redact logs, restrict backups, and obtain legal review.	CERA and Dev 3
Single-server failure	Daily off-server database dump, Hetzner backup, immutable images, documented rebuild, tested staging restore, and deletion protection.	Dev 3
Third-party outage	Durable integration jobs, idempotency, retries with backoff, dead-letter visibility, reconciliation, and customer-safe failure handling.	Dev 2
Release regression	Required checks, staging UAT, immutable image promotion, pre-deploy backup, smoke test, monitoring, and recorded rollback.	Dev 3

24 Research basis

The delivery baseline was reviewed against the following official documentation on 17 September 2026. Vendor interfaces and plan entitlements can change, so Dev 3 must recheck implementation-specific details when the repository and accounts are configured.

R1 GitHub rulesets and required reviews
<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>

R2 GitHub deployments and environments
<https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>

R3 GitHub Actions secure use
<https://docs.github.com/en/actions/reference/security/secure-use>

R4 GitHub publishing Docker images
<https://docs.github.com/en/actions/tutorials/publish-packages/publish-docker-images>

R5 Next.js deployment
<https://nextjs.org/docs/app/getting-started/deploying>

R6 Payload production deployment
<https://payloadcms.com/docs/production/deployment>

R7 Payload storage adapters and Cloudflare R2
<https://payloadcms.com/docs/upload/storage-adapters>

R8 Payload drafts and versions
<https://payloadcms.com/docs/versions/drafts>

R9 Vendure worker and job queue
<https://github.com/vendurehq/vendure/blob/master/docs/docs/guides/developer-guide/worker-job-queue/index.mdx>

R10 Authentik Docker Compose installation
<https://docs.goauthentik.io/install-config/install/docker-compose>

R11 Authentik OAuth 2.0 and OpenID Connect provider
<https://docs.goauthentik.io/add-secure-apps/providers/oauth2>

R12 Cloudflare R2 presigned URLs
<https://developers.cloudflare.com/r2/api/s3/presigned-urls/>

R13 Caddy automatic HTTPS
<https://caddyserver.com/docs/automatic-https>

R14 GlitchTip self-hosted installation
<https://glitchtip.com/documentation/install/>

R15 Zoho CRM V8 API reference
<https://www.zoho.com/crm/developer/docs/api/v8/api-references.html>

R16 Zoho CRM OAuth overview
<https://www.zoho.com/crm/developer/docs/api/v8/oauth-overview.html>

R17 Resend idempotency keys
<https://resend.com/changelog/idempotency-keys>

R18 Hetzner backups and snapshots
<https://docs.hetzner.com/cloud/servers/backups-snapshots/overview/>

R19 PostgreSQL backup and restore
<https://www.postgresql.org/docs/17/backup.html>

R20 Docker Compose in production
<https://docs.docker.com/compose/how-tos/production/>

R21 WCAG 2.2
<https://www.w3.org/TR/WCAG22/>

R22 OWASP Top 10 2025
https://top10.owasp.org/2025/0x00_2025-Introduction/

Product approval

Role	Name	Signature or GitHub approval	Date
CERA Product Owner			
Content and Clinical Approver			
Technical Release Approver			